

ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL
FACULTAD DE INGENIERÍA EN ELECTRICIDAD Y COMPUTACIÓN

TAREA #6 — GESTIÓN DE ENERGÍA EN EL ESP32

Nombre: Fernando Vélez

Fecha de entrega: 08 de agosto de 2026

1. Descripción del ejercicio

Firmware para ESP32 desarrollado con **ESP-IDF** sobre **PlatformIO** (lenguaje C, en Visual Studio Code) que alterna entre una fase de trabajo activo y una fase de reposo en bajo consumo, con dos fuentes de despertar y señalización por LED y por puerto serial.

Fase	Comportamiento
Proceso activo (10 s)	CPU a plena velocidad, LED rojo parpadeando cada 500 ms
Reposo (15 s o botón)	<code>esp_light_sleep_start()</code> : la CPU se pausa
Reactivación	Se reporta el origen: BOTÓN o TEMPORIZADOR

Enlaces de la entrega:

- Repositorio: https://github.com/FernandoVB01/Tarea_06_Sistemas-Embebidos
- Simulación en Wokwi: <https://wokwi.com/projects/471758513152881665>
- Video de funcionamiento: <https://youtube.com/playlist?list=PLKeUXsXQwJig>

2. Conexiones

Señal	GPIO	Componente	Nota
LED PROCESO	2	LED rojo + 220 Ω	indica trabajo activo
LED ESPERA	4	LED amarillo + 220 Ω	marca la entrada en reposo
BOTÓN	5	Pulsador a GND	pull-up interno, activo en bajo
GND	—	—	cátodos de los LEDs y pulsador

El pulsador no lleva resistencia externa: se habilita el pull-up interno del ESP32, de modo que el pin lee 1 en reposo y 0 al pulsar.

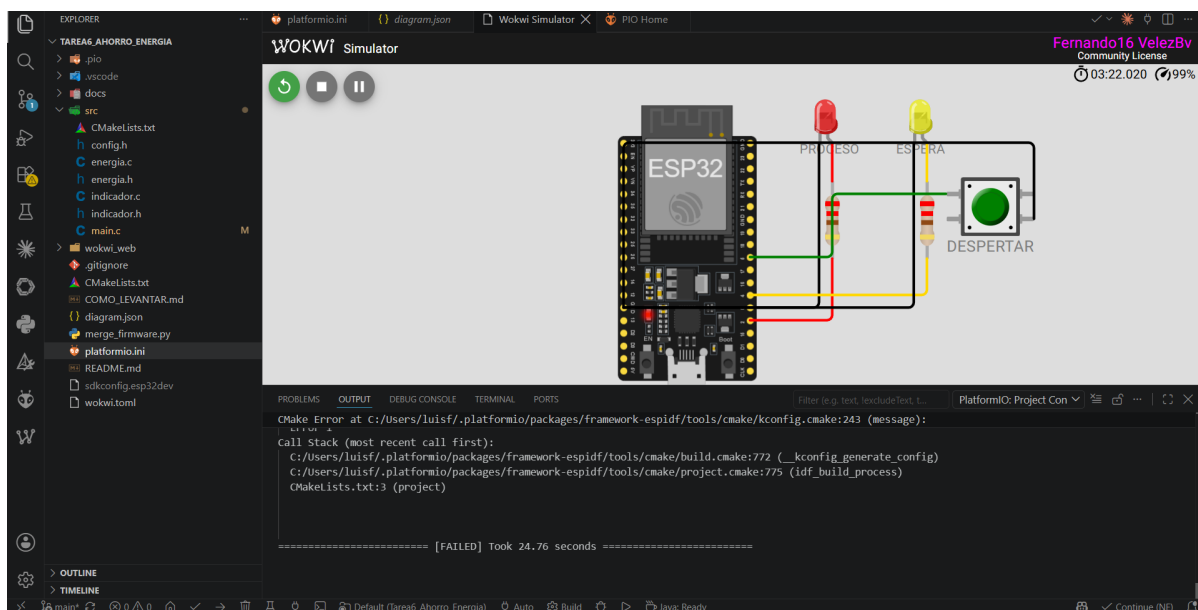


Figura 1: Circuito montado en el simulador de Wokwi, dentro de Visual Studio Code

3. Funcionamiento

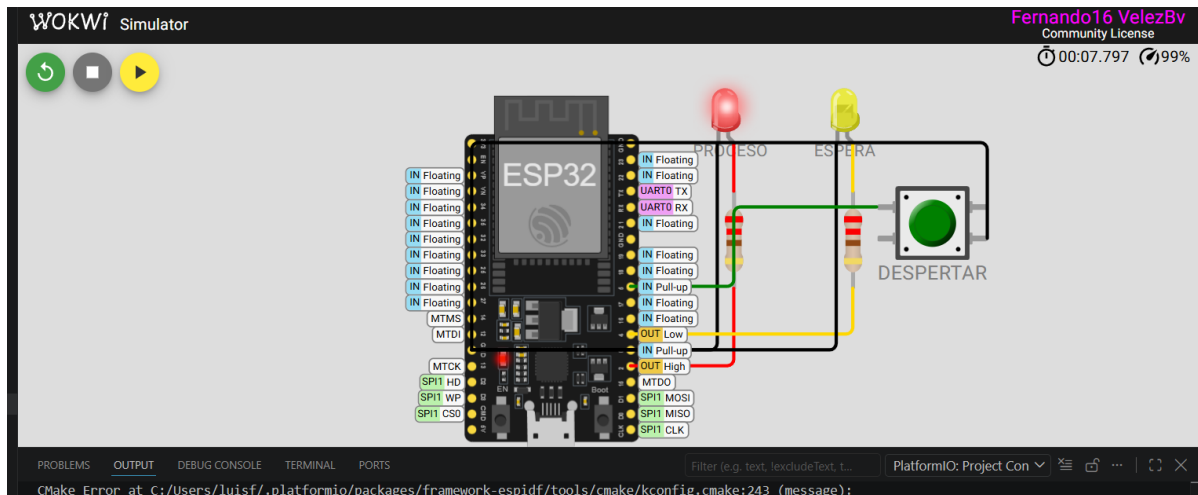


Figura 2: Fase de proceso activo. El simulador muestra el estado de cada pin: **OUT High** en GPIO2 (LED PROCESO encendido), **OUT Low** en GPIO4 y **IN Pull-up** en GPIO5, a la espera del pulsador

La anotación de los pines confirma que el firmware está gobernando el hardware: GPIO2 en alto durante el trabajo, GPIO4 en bajo, y GPIO5 configurado como entrada con pull-up interno. Al agotarse los 10 segundos ambas salidas pasan a bajo y el chip entra en light sleep.

4. Implementación física

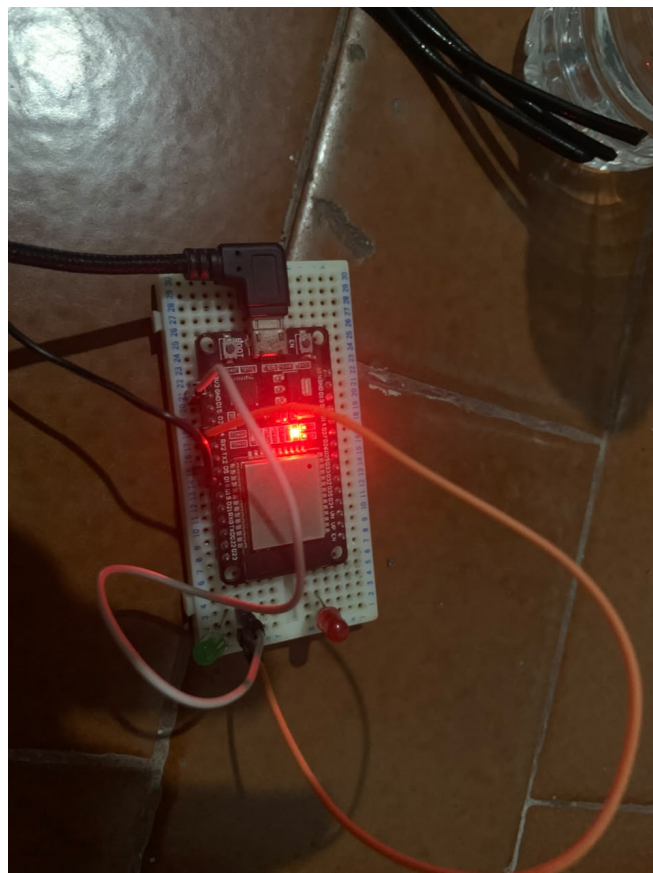


Figura 3: Montaje sobre protoboard: ESP32 alimentada por USB con el LED de proceso encendido durante la fase activa

5. Análisis del comportamiento

¿Cuándo debe dormir el sistema? Cuando no hay trabajo útil pendiente. En un nodo IoT el ciclo real es *medir* → *transmitir* → *dormir*, y la fase de reposo ocupa más del 99 % del tiempo. La autonomía depende mucho más del consumo en reposo que del consumo en actividad.

¿Qué eventos provocan el despertar? Dos, combinadas para dar el patrón habitual «*despierta cada 15 s, o antes si pasa algo*»:

- `esp_sleep_enable_timer_wakeup()` — temporizador RTC: despertar periódico y predecible, base del muestreo por intervalos.
- `gpio_wakeup_enable()` — pin externo: despertar por evento asíncrono, sin gastar energía sondeando el pin.

¿Cómo se comporta antes y después? Tras el light sleep la ejecución continúa en la línea siguiente y la RAM conserva su contenido: es reanudación, no reinicio. Por eso el contador de ciclos es una variable normal y no necesita memoria RTC.

6. Conclusiones

- **El ahorro viene de pausar la CPU, no de apagar el indicador.** Una espera implementada con un bucle de retardo produce el mismo comportamiento visible —el LED se apaga y la consola anuncia el reposo— pero el consumo no baja, porque el núcleo sigue ejecutando instrucciones. La diferencia solo aparece al invocar `esp_light_sleep_start()`, que lleva el consumo de unos 40 mA a unos 0,8 mA. El comportamiento observable y el comportamiento energético son cosas distintas, y solo el segundo es el objeto de esta práctica.
- **La elección del pin condiciona qué modo de despertar existe.** La fuente `ext0` exige un GPIO con función RTC, porque en deep sleep el dominio digital está apagado y solo el subsistema RTC sigue vigilando. GPIO5 no tiene esa función, de modo que aquí el despertar externo se implementó con `gpio_wakeup_enable()`, válido en light sleep porque ese dominio permanece alimentado. Migrar este mismo firmware a deep sleep obligaría a reubicar el pulsador en un pin RTC.
- **La placa de desarrollo impone un piso de consumo que el datasheet no muestra.** Las cifras del fabricante corresponden al chip desnudo. En una DevKit conviven el regulador AMS1117 y el conversor USB-serial, ambos alimentados de forma permanente, y su corriente de reposo domina la lectura del multímetro. Alcanzar el consumo nominal es un problema de diseño de hardware, no solo de firmware.

7. Recomendaciones

- **Apagar explícitamente todo lo que consuma antes de dormir, y comprobarlo midiendo.** LEDs, sensores alimentados por GPIO y pull-ups activos anulan el ahorro sin dejar ningún rastro en el código: el programa parece correcto y el consumo no baja. En este firmware el LED de espera se apaga justo antes de entrar en el sueño por esa razón. La única verificación fiable es el amperímetro.
- **Fijar la versión del toolchain en el control de versiones.** Durante el desarrollo, la versión de ESP-IDF instalada por defecto generaba un binario que no arrancaba en el simulador, sin ningún mensaje de error que lo delatara. Anclar la versión en `platformio.ini` hace el proyecto reproducible y evita que un entorno actualizado rompa un firmware que ya funcionaba.